Bidirekcioni Dajkstrin algoritam

Igor Milovanović

Uvod

- Jedan od najpoznatijih problema u teoriji grafova jeste pronalazak najkraćeg puta između dva čvora. Pod pojmom ,,najkraći put" možemo podrazumevati put koji sadrži najmanji broj grana, ili ako radimo na težinskim grafovima, put sa najmanjom *cenom.* U ovom radu, bavićemo se problemom pronalaska najkraćeg puta u smislu cene datog puta.
- Jedan od najpoznatijih algoritama koji se bavi traženjem najkraćeg puta u težinskom grafu jeste **Dajkstrin algoritam**.
- Ovaj algoritam dobio je naziv po naučniku Edsgeru W. Dijkstri, koji je ovaj algoritam objavio 1959. godine.
- Dajkstra nam daje jedan vrlo efikasan algoritam, koji izračunava najkraći put od određenog čvora u grafu, do svih drugih čvorova.
- Problem koji se prirodno nameće jeste pronalazak najkraćeg puta u grafu između dva tačno određena čvora datog grafa.
- Dajkstrin algoritam je primenjiv i na usmerenim I neusmerenim gragovima.

Dajkstrin algoritam

- Neka je G=(V,E) **težinski graf**, pri čemu je V-skup čvorova tog grafa, E-grane tog grafa čiji su elementi predstavljeni uređenom trojkom (u,v,t) gde su $u, v \epsilon V$, a t je težina te grane koja je pozitivna.
- Težinu grane e=(u,v,t) označavaćemo sa $l(uv)$, od reči *length* tj dužina.
- **Dajkstrin algoritam** nam daje postupak za pronalazak najkraćeg puta i njegove dužine, od datog početnog čvora $u \in V$ do svakog čvora $v \in V$ datog grafa G. Ukoliko neki čvor v nije dostižan iz čvora u, tada je na kraju algoritma, dužina najkraćeg puta do njega postavljena na ∞.
- Tokom rada algoritama, čuvamo informacije o trenutnim **dužinama najkraćih poznatih puteva** do svakog čvora grafa G. Trenutnu dužinu najkraćeg poznatog puta od čvora u do čvora v označavaćemo sa $\boldsymbol{d(u,v)}$**.**
- Na početku algoritma, postavljamo vrednosti trenutno najkraćih puteva na 0, za početni čvor, i ∞ do svih drugih čvorova grafa G.
- Tokom rada algoritma, za svaki čvor grafa čuvamo i informaciju o tome da li je do tog čvora pronađen **najkraći mogući put**. Ukoliko je do nekog čvora pronađen najkraći mogući put reći ćemo da je taj čvor **posećen**, a u suprotnom je **neposećen**.

- 1: Od svih čvorova grafa G biramo onaj koji je neposećen i do koga je vrednost dužine trenutnog najkraćeg poznatog puta minimalna I različita od ∞. Taj čvor označavamo kao posećen. Neka je to čvor v.
- Prolazimo kroz skup svih suseda čvora v. Kada naiđemo na suseda čvora v koji je neposećen, proveravamo da li je pronađen kraći put od trenutno najkraćeg poznatog puta do tog suseda, to jest da li je najkraći put do čvora v plus težina grane do tog suseda, manji od trenutno najkraćeg poznatog puta do tog suseda. Ukoliko jeste, ažuriramo novu vrednost za suseda. Označimo suseda sa s.
- U navedenim oznakama proveravamo da li je:
  $$d(u,v) + l(vs) < d(u,s)$$ i ukoliko jeste:
  $$d(u,s) = d(u,v) + l(vs)$$
- Ponavljamo postupak 1, sve dok takvi čvorovi postoje. Ukoliko više takvih čvorova ne postoji algoritam se završava.
- Dajkstrin algoritam nam garantuje sledeće tvrdnje.
  1. Do svakog čvora v grafa G koji je **dostižan** iz početnog čvora u će na kraju algoritama biti određena dužina najkraćeg mogućeg puta od čvora u do čvora v. Ukoliko je čvor v **nedostižan** iz u dužina najkraćeg puta do njega će biti ∞.
  2. U trenutku kada čvor označimo da je **posećen**, dužina najkraćeg mogućeg puta do njega je poznata. Niti jedan novi put koji pronađemo, ne može biti kraći.

Implementacija Dajkstrinog algoritma

- Dajkstrin algoritam se može implemetirati sa nekoliko pomoćnih sturktura. Posmatrajmo graf G=(V,E).
- Neka je n broj čvorova grafa G, to jest $n = |V|$.
- Neka je m broj grana grafa G, to jest $m = |E|$
- Grane grafa ćemo predstaviti preko **liste susedstva.**
- Informacije o tome da li je čvor posećen ili ne, čuvaćemo u jednom nizu logičkih vrednosti dužine n. Ukoliko je čvor neposećen vrednost je **false**, a ukoliko je posećen vrednost je **true**. Na početku inicijalizujemo ceo niz na false. Ovaj niz ćemo nazivati **visited** ili **settled.**
- Informacije o trenutno najkraćem poznatom putu do svakog čvora grafa G, čuvamo u nizu dužine n. Na početku vrednost za početni čvor inicijalizujemo na 0, dok za sve druge inicijalizujemo na ∞. Ovaj niz ćemo nazivati **distances.**
- Informacije o kandidatima za nastavak pretrage ćemo čuvati u jednom redu sa prioritetom, koji je organizovan tako da element sa najvećim prioritetom onaj do koga je trenutno poznati najkraći put minimalan. Taj red ćemo označavati sa **queue.**
- Ove notacije i oznake će nam biti važne u opisivanju narednih modifikacija Dajkstrinog algoritma.

Najkraći put između dva čvora grafa

- Problem koji se prirodno nameće nakon Dajkstrinog algoritma je kako da pronađemo najkraći mogući put između data dva čvora u,v grafa G. Dajkstrin algoritam rešava problem pronalaska svih najkraćih puteva jednog, do svih drugih čvorova tog grafa.
- Ovaj problem jeste rešiv pomoću Dajkstrinog algoritma. Ukoliko jedan od datih čvorova proglasimo za početni i pokrenemo algoritam iz njega, nesumljivo ćemo na kraju algoritama znati dužinu najkraćeg puta između ta dva čvora.
- **Međutim možemo i bolje**: Dajkstrin algoritam nam **daje garanciju** da u trenutku kada čvor proglasimo za posećen, trenutni poznati najkraći put do njega je sigurno i najkraći mogući put do tog čvora. Označimo u kao početni čvor algoritma. U trenutku kada označimo v kao posećen čvor, algoritam možemo zaustaviti, jer je najkraći mogući put do njega već poznat i time potencijalno uštedeti vreme **jer nećemo razmatrati čvorove koji su na većoj udaljenosti od čvora u**.
- Iako ova modifikacija može značajno ubrzati postupak, intuicija nalaže da sigurno mora postojati efikasnije rešenje ovog problema.

- Ideja koja se prirodno nameće jeste da pretragu krenemo iz oba čvora čekajući da se njihove pretrage sretnu ***na pola puta.***
- Ovaj problem možemo posmatrati kao problem pronalaska najkraćeg puta od **izvora** do **cilja**. To odgovara situaciji ukoliko je reč o usmerenom grafu, i mi tražimo najkraći usmereni put od izvora do cilja u datom grafu. Ukoliko je reč o neusmerenom grafu, za data dva čvora možemo samo odabrati da je jedan izvor, a drugi je cilj, te implementacija, korektnost i složenost neće zavisiti od našeg izbora.
- Prvi naučni rad koji se bavio dvosmernim pristupom Dajkstrinog algoritma je obljavljen samo sedam godina nakon što je Dajkstra objavio svoj naučni rad.
- Bidirekcioni (dvosmerni) Dajkstrin algoritam za najkraći put prvi put je predložio matematičar **George Dantzig** 1963. godine (u formi osnovne ideje), dok je precizan i korektan opis algoritma prvi objavio **T. A. J. Nicholson** 1966. godine.
- Od tada ovaj algoritam je dobio naziv **Bidirekcioni Dajkstrin algoritam**.

Bidirekcioni Dajkstrin algoritam

- Osnovna ideja ovog algoritma jeste da se pretraga krene u dva smera, i od izvora ka cilju, i od cilja ka izvoru i da se nastoji da se te dve pretrage sretnu *na pola puta*.
- Ovaj algoritam se relativno lako može konstruisati i implementirati jer svoju korektnost crpi iz jednosmernog Dajkstrinog algoritma.
- Glavni problemi koji treba rešiti su **prepoznavanje da su se dve pretrage ,,srele”** , **ažuriranje dužine trenutno poznatog najkraćeg puta od izvora do cilja** i **uslov zaustavljanja.**
- Pod uslovom zaustavljanja smatramo uslov koji mora da bude ispunjen da bi trenutno najkraći poznat put od izvora do cilja bio i najkraći mogući put od izvora do cilja.
- Posmatrajmo sada **neusmereni graf G=(V,E).**
- Treba pronaći najkraći put između dva čvora grafa G. Odaberimo jedan koji ćemo zvati izvor i označiti gas a **s** (**source**), a drugi ćemo zvati cilj i označiti gas a **t** (**target**).
- Pretragu iz izvora i iz cilja modeliramo na isti način kao I kod jednosmernog Dajkstrinog algoritama. Obrađujemo jedan po jedan čvor naizmenično iz pretrage pokrenute iy izvora i pretrage pokrenute iz cilja.

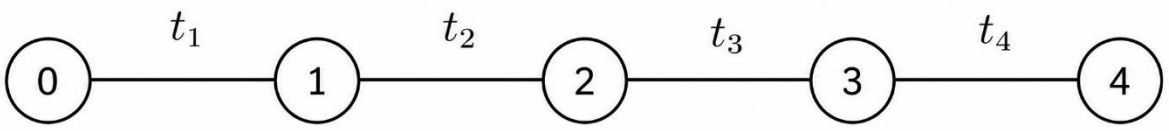
- Proces ćemo objasniti kroz prolazak kroz pretragu izvora, te se pretraga iz cilja ponaša na identičan način.
- Neka trenutno obrađujemo susede čvora u koji smo izabrali na način opisan u koraku 1 Dajkstrinog algoritma. Ukoliko naiđemo na nekog njegovog suseda v, **koji je posećen od strane pretrage pokrenute iz cilja**, to znači da smo pronašli jedan put od izvora do cilja, i njegova dužina nam je poznata.
- Naime, pošto je čvor u posećen od strane izvora, najkraći mogući put od izvora do čvora u nam je poznat. Slično poznat je najkraći mogući put od cilja do čvora v. Pošto je v sused od u, **onda je nama poznat jedan mogući put od izvora do cilja, jer se on sastoji od puta od izvora do čvora u, preko grane uv, i puta od čvora v do cilja.**
- Zapravo, svi mogući putevi između dva **razna čvora** jednog grafa su zapravo ovog oblika, jer nije isključeno da je $s = u$ ili $v = t$.
- Stoga, **najkraći mogući put od izvora do cilja nam predstavlja minimalan, u smislu cene, takav put**.
- Time smo rešili problem prepoznavanja trenutka kada se dve pretrage ,,sretnu".
- Mi bismo mogli pustiti da se obe pretrage izvrše do kraja, pamteći I ažurirajući najkraći put od izvora do cilja, ali tada bismo zapravo samo obe pretrage izvršili do kraja, te bi vreme potrebno za izvršavanje nepotrebno dupliralo. Stoga nam je potreban korektan uslov zaustavljanja.

- **Najkraći trenutno poznati put od izvora do cilja ćemo označiti sa μ.**
- Neka je u čvor koji trenutno obrađujemo pretragom iz izvora, a v iz cilja. Ukoliko važi da je
$$\mu \leq d(s,u) + d(t,v)$$
Tada će važiti i da je $\mu < d(s,u) + l(uv) + d(t,v)$
Algoritam možemo završiti I proglasiti μ za dužinu najkraćeg mogućeg puta od izvora do cilja.
- Zbog čega ovo važi? Pošto je udaljenost od izvora do u sigurno kraća ili jednaka od udaljenosti izvora do svih drugih neposećenih čvorova izvora (to nam garantuje način izbora čvora u), i udaljenost od cilja do v sigurno kraća ili jednaka od udaljenosti izvora do svih drugih neposećenih čvorova cilja (to nam garantuje način izbora čvora v), tada će za sve neposećene čvorove izvora u' i sve neposećene čvorova cilja v' važiti sledeće: Ukoliko postoji grana od u' do v' tada će zasigurno biti:
$$\mu < d(s,u') + l(u'v') + d(t,v')$$
- Ovaj uslov nam garantuje da novi putevi koje ćemo možda pronaći, ne mogu da budu kraći od trenutnog poznatog najkraćeg puta od izvora do cilja, te algoritam možemo završiti i μ proglasiti za najkraći put od izvora do cilja.
- Stoga ukoliko nadjemo put za koji važi
$$\mu > d(s,u) + l(uv) + d(t,v)$$
Tada je to najkraci poznati put od izvora do cilja i ažuriramo $\mu = d(s,u) + l(uv) + d(t,v)$

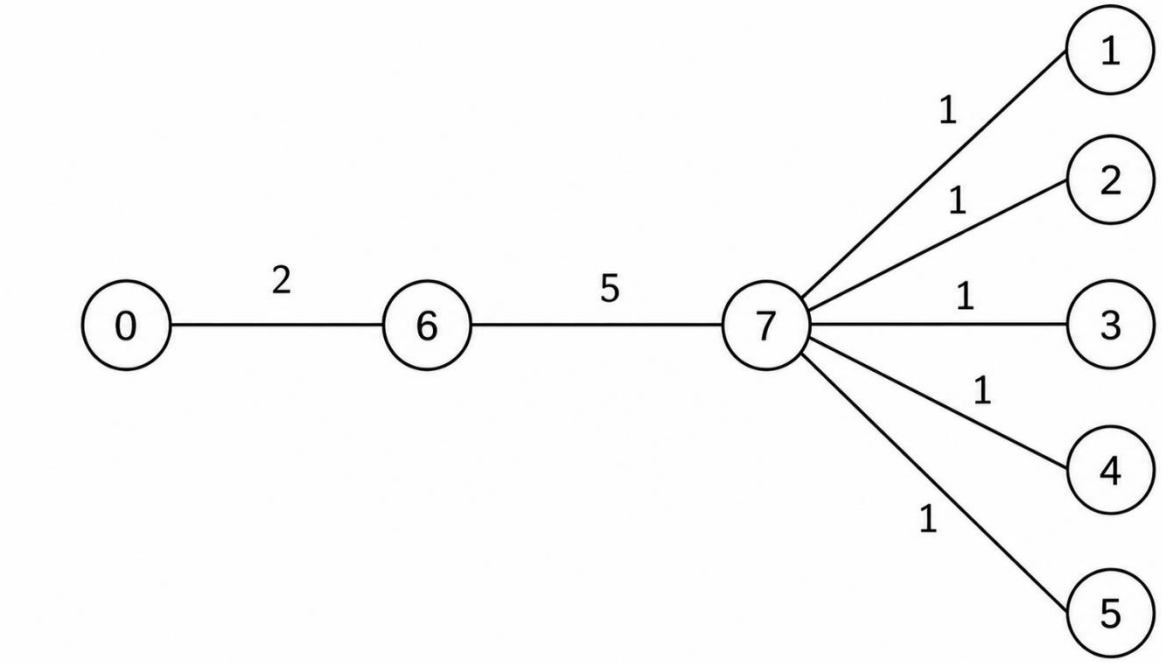
- Time smo rešili i preostala dva problema konstrukcije Bidirekcionog Dajkstrinog algoritma.
- Algoritam funkcionise tako što naizmenično obrađuje jedan čvor iz pretage iz izvora, pa jedan čvor iz pretrage iz cilja.
- Ukoliko imamo **usmereni težinski graf**, algoritam Bidirekcione Dajkstrine pretrage lako prilagođavamo tako što imamo jasno definisan izvor i cilj, te tražimo **usmeren put** od izvora do cilja.
- Kada vršimo pretragu od cilja, grane posmatramo kao suprotno usmerene, gde efektivno nalazimo najkrace puteve od svakog drugog čvora do cilja.
- Implementacija Bidirekcione dijikstre se vrši lako, samo će nam trebati po 2 strukture za one iz implementacije jednosmernog Dajkstrinog algoritma.
- Imaćemo logičke nizove **visited_from_source**, **visited_from_target** za potrebe pamćenja čvorova koji su posećeni.
- Imamo brojevne nizove **distance_from_source** i **distance_from_target** za pamćenje trenutno poznate najkraće udaljenosti.
- Imamo 2 reda sa prioritetima **queue_from_source** i **queue_from_target**, za odabir sledećih čvorova za obradu.
- I ukoliko se radi o usmerenom grafu, imamo pored liste povezanosti **neighbours** i **neighbours_reverse** za suprotno usmerene grane.

Efikasnost bidirekcionog Dajkstrinog algoritma

- U praksi, za rešavanje problema pronalaska najkraćeg puta između dva čvora grafa, dvosmerni pristup se pokazao mnogo efikasniji od jednomsernog pristupa.
- Pošto se oba pristupa implementiraju na isti način, koriste iste strukture, glavna složenost ova dva algoritma leži u broju obrađenih grana tokom algoritma i složenosti održavanja niza prioriteta. Stoga tu leži i razlika u njihovoj efikasnosti.
- U nekim slučajevima jednosmeran i dvosmeran pristup će se ponašati na isti način. Na primer u primeru sa slike 1, ukoliko su dva čvora 0 i 4 oba pristupa će obraditi sve četiri grane grafa. Ukoliko je graf *linearan,* sa m grana, a čvorovi su krajnji, oba pristupa će obraditi m grana.
  Slika 1:



- U nekim situacijama, jednosmerni pristup može čak biti i efikasniji od dvosmernog pristupa. Posmatrajmo na primer sliku broj 2.
  Slika 2:



- Ukoliko treba da pronađemo najkraci put od čvora 0 do čvora 7, jednosmerni algoritam bi pregledao samo 2 grane, dok bi bidirekcioni pogledao 8 grana. Naime, pošto se obrađuju jedan po jedan čvorovi naizmenično iz izvora i cilja, bidirekcioni pristup bi iz čvora 0 pregledao granu (0,6) , pa bi onda iz cvora 7 pregledao svih 6 grana iz čvora 7. Onda bi sledeći čvor pretrage iz izvora bio 6, pa bi našao put do 7 dužine 7. Iz cilja bi obradili čvor 1 koji ima jednu granu. Tek onda, kada je sledeći čvor iz izvora 7, zaključili bismo da smo pronašli najkraći put.

- Međutim ovi primeri su veštački napravljeni. Efikasnost bidirekcionog pristupa možemo pokazati na sledeći način. Neka su izvor i cilj čvorovi koji su na udaljenosti k. Zamislimo graf kao skup tačaka u ravni ravnomerno raspoređenih.
- Da bismo jednosmernim pristupom došli od izvora do cilja, mi bismo morali da obiđemo svaki čvor unutar kruga poluprečnika k. Znači površina datog kruga bi bila $k^2\pi$.
- Ukoliko pristupimo dvosmernim algoritmom, da bi se ova dva kruga dotakla, potrebno je da obiđemo svaki čvor unutar 2 kruga poluprečnika k/2. Površina koju ta dva kruga zauzimaju je $2 * \left(\frac{k}{2}\right) * \left(\frac{k}{2}\right) * \pi = \frac{1}{2}k^2\pi$
- Ukoliko su čvorovi ravnomerno raspoređeni bidirekcionim pristupom bismo obišli dva puta manje čvorova. Pritom,
- Ovu efikasnost možemo predstaviti i na malo drugačiji način. Pojednostavićemo graf. Neka je stepen svakog čvora grafa jednak b, i neka je težina svake grane grafa jednaka 1. Neka je dužina najkraćeg puta od izvora do cilja jednaka k.
- Jednosmerni pristup: Mi prvo moramo da obiđemo sve čvorove na udaljenosti 1 od izvora, njih ima b. Nakon toga, treba da obidjemo sve one na udaljenosti 2 od izvora, a njih ima b^2. Čvorova koji su na udaljenosti k ima b^k. Te je ukupan broj čvorova koji moramo da obiđemo $1 + b +$

$b^2 + \cdots + b^k = \frac{b^{k+1}-1}{b-1}$. Svaki od tih čvorova ima b suseda, te će broj obiđenih grana biti $b * \frac{b^{k+1}-1}{b-1} \approx b^{k+1}$

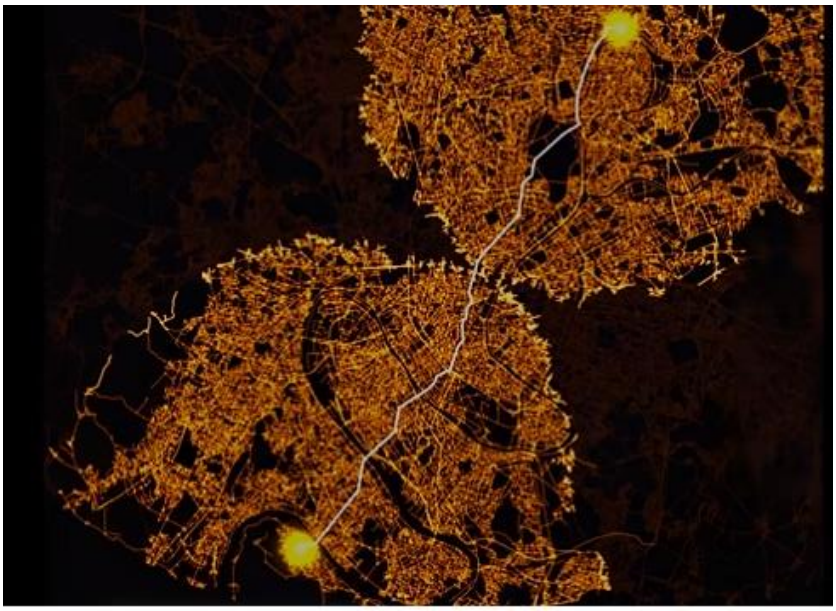
- Ukoliko bi primenili bidirekcioni pristup, morali bi obići sve koji su na udaljenosti maksimalno k/2 od oba čvora. Te bi broj posećenih čvorova bio

$$2\left(1 + b + b^2 + \cdots + b^{\frac{k}{2}}\right) = 2 * \frac{b^{\frac{k}{2}+1} - 1}{b - 1}$$

- Pošto svaki čvor ima b grana obradili bi

$$2b * \frac{b^{\frac{k}{2}+1}-1}{b-1} \approx 2b^{\frac{k}{2}+1}$$

- Možemo videti da se broj obrađenih grana **značajno smanjio**. Međutim i ovaj primer je veštački napravljen, ali nije mnogo daleko od realne primene, kada tražimo najkraći put između dve tačke na mapi.



Bidirekcioni Dajikstrin algoritam- novi pristup

- Problem pronalaska najkraćeg puta između dva data čvora nekog grafa korišćenjem dvosmernog Dajkstrinog algoritma dobija novo i vrlo značajno rešenje 2025. godine.
- Grupa naučnika koju čine **Bernhard Haeupler**, **Richard Hladík**, **Václav Rozhoň**, **Robert E. Tarjan** i **Jakub Tětek** objavila je 2025. godine naučni rad pod naslovom „Bidirectional Dijkstra's Algorithm is Instance-Optimal".
- U ovom naučnom radu, oni su bidirekcioni Dajkstrin algoritam implementirali na malo drugačiji način, i za njihovu verziju algoritma dokazali veoma važnu teorijsku tvrnju.
- Glavna razlika između njihovog algoritma i prethodno navedenog algoritma bidirekcione dajkstre, je u tome da dve pretrage ne alterniraju po obradi čvorova, već po obradi(relaksaciji) po jedne grane pretrage iz izvora i pretrage iz cilja.
- Naime nek se trenutno obrađuje čvor u_s iz izvora. Neka čvor u_s ima susede, koji su redom u listi čvorova navedeni na ovaj način $v_1, v_2, \dots v_m$. Neka se sa obradom stiglo do nekog čvora v_i, tj grane $u_s v_i$, ona je prva iz liste susedstva koja nije obrađena.

- Nek se trenutno obrađuje čvor u_t iz pretrage iz cilja. Neka čvor u_t ima susede, koji su redom u listi čvorova navedeni na ovaj način $r_1, r_2, \dots r_o$. Neka se sa obradom stiglo do nekog čvora r_j, tj grane $u_t r_j$, ona je prva iz liste susedstva koja nije obrađena.
- Pretpostavimo da je na redu pretraga iz izvora. **Relaksiramo granu $\boldsymbol{u_s v_i}$**. Pod pojmom relaksacije grane podrazumevamo provere a) i b).
- a) Proveravamo da li je čvor $\boldsymbol{v_i}$ posećen iz izvora. Ukoliko nije proverimo da li je to novi najkraći put od čvora s (izvora) do čvora $\boldsymbol{v_i}$. Ukoliko jeste, ažuriramo vrednost trenutno najkraćeg poznatog puta do čvora v_i.
- b) Proveravamo da li je čvor $\boldsymbol{v_i}$ posećen iz cilja. Ukoliko jeste, proveravamo da li smo pronašli novi najkraći put od izvora do cilja, na isti način kao u prehodnoj implementaciji.
- Kada smo završili relaksaciju grane iz izvora, relaksiramo sledeću granu iz pretrage iz cilja, tj granu $u_t r_j$ na analogan način. Na ovaj način alterniramo između relaksacije grana iz izvora i cilja, tj naizmenično relaksiramo po jednu granu.
- Ukoliko smo došli do kraja liste susedstva, na identičan način kao u prošloj implementaciji, biramo sledeći čvor za obradu, i označavamo njega kao posećenog.
- Tada proveravamo **uslov zaustavljanja** algoritma, koji je identičan uslovu zaustavljanja iz prethodne implementacije.

- Implementacija data u naučnom radu iz 2025. godine:

Algorithm 2: Instance-Optimal Bidirectional Dijkstra's Algorithm

Input: Graph $G(V, E)$, source vertex s, target vertex t

$\mu \leftarrow +\infty$; `// Length of the shortest path that we have found so far`

$e_{\text{mid}} \leftarrow \perp$; `// Middle edge of the shortest path that we have found so far`

$u_s \leftarrow s, u_t \leftarrow t$; `// Vertices currently being explored in the two executions`

Initialize forward search from s on G and backward search from t on G with edges reversed;

while *neither of the two executions has terminated* **do**

  Alternate between relaxing one edge in the Forward and Backward algorithm;

end

$uv \leftarrow e_{\text{mid}}$;

$P \leftarrow$ "shortest su-path according to forward execution" + e_{mid} + "shortest vt-path according to backward execution";

return P;

Function `Forward_algorithm`**:**

  $\hat{d}(s, \cdot) \leftarrow +\infty$; $\hat{d}(s, s) \leftarrow 0$;

  Open s;

  while *an open vertex exists* **do**

    Let u be the open vertex with the smallest $\hat{d}(s, u)$;

    Close u;

    Set $u_s \leftarrow u$;

    if $\hat{d}(s, u_s) + \hat{d}(u_t, t) \geq \mu$ **then**

      terminate the whole algorithm;

    end

    for v *a forward neighbor of* u **do**

      if v *is not closed* **then**

        $\hat{d}(s, v) \leftarrow \min(\hat{d}(s, v), \hat{d}(s, u) + \ell(uv))$;

      end

      if v *is closed in the backward execution and* $\hat{d}(s, u) + \ell(uv) + \hat{d}(v, t) < \mu$ **then**

        $\mu \leftarrow \hat{d}(s, u) + \ell(uv) + \hat{d}(v, t)$;

        $e_{\text{mid}} \leftarrow uv$

      end

    end

  end

Function `Backward_algorithm`**:**

  Analogous to Forward algorithm with the roles of *forward* and *backward* flipped;

Teorijski značaj novog pristupa

- Veliki teorijski značaj ovog naučnog rada je jer autori dokazuju da je ovako modeliran algoritam bidirekcione Dajkstre **instance-optimalan.**
- Da bismo objasnili ovaj pojam, i značaj ove tvrdnje, moramo uvesti nekoliko termina.
- **Mera složenosti algoritma** je funkcija koja svakom ulazu x pridružuje broj ili količinu resursa koje algoritam utroši pri rešavanju tog ulaza, pri čemu se unapred definiše koji se resurs meri (npr. broj operacija, vreme izvršavanja, memorija ili broj upita).
- Ukoliko pod merom složenosti nekog algoritma podrazumevamo broj upita, radi se o složenosti upita, koju nazivamo **query complexity.**
- Upit nad grafom možemo posmatrati svaki pojedinačan pristup nekoj informaciji grafa. Na primer upit je pristup sledećem elementu iz liste povezanosti određenog čvora.
- Kako je glavna operacija zapravo pristup (upit) za jednu granu i obrada te grane autori dokazuju optimalnost ovog algoritma po broju grana koje su obrađene.
- **Adjacency-list model** je model predstavljanja i pristupa grafu u kome je graf predstavljen listom susedstva, tako da se za svaki čvor čuva lista njegovih susednih čvorova, dok

algoritam informacije o grafu dobija **isključivo putem definisanih upita (query-ja) nad tim listama.**

- **Multigraf** je graf u kojem između ista dva čvora može postojati više različitih grana, odnosno više grana koje povezuju isti par čvorova. U multigrafu se takođe mogu dozvoliti petlje, tj. grane koje povezuju čvor sa samim sobom, u zavisnosti od usvojene definicije.
- Navodimo definiciju instance-optimalnosti za algoritme nad grafovima, gde kao meru složenosti određenog algoritma posmatramo kao query complexity.
- Neka je A algoritam koji rešava problem P. Kažemo da je A **instance-optimalan** ako za svaki korektan algoritam L koji rešava problem P postoji konstanta c>0 takva da za svaki graf G važi:

$$\boldsymbol{Q_A(G) \leq c * Q_L(G)}$$

gde je $Q_A(G)$ broj upita koje algoritam A izvrši nad grafom G, a $Q_L(G)$ broj upita koje algoritam L izvrši nad istim grafom G.

- Problem pronalaska najkraćeg puta od izvora do cilja ćemo nazivati **st-problem najkraceg puta**(source-target path).
- Autori dokazuju sledeću tvrdnju. Neka je G=(V,E) **usmereni/neusmereni** težinski **multigraf** čije su sve grane pozitivne težine. Neka je multigraf predstavljen **adjacency-list modelom**: Algoritam 2 je instance-optimalan algoritam za st-problem najkraćeg puta, po broju grana kojima pristupa.

- Dokaz ovog tvrđenja nećemo navoditi zbog njegove kompleksnosti, ali ćemo prokomentarisati njegovu važnost.
- Ova tvrdnja nam zapravo govori da će bilo koji drugi korektan algoritam, u rešavanju st-problema najkraćeg puta može pristupiti samo konstantanom broju puta manje grana, i ta konstanta ne zavisi od grafa nad kojim se algoritam primenjuje, pa je ona zapravo konstantan faktor-O(1).
- Ovo je vrlo jaka tvrdnja jer nam govori da će ovako definisan bidirekcioni Dajkstrin algoritam, uvek biti vrlo blizu optimalnom algoritmu za bilo koji (multi)graf G, po broju grana kojim pristupa.
- Ukoliko govorimo o efikasnosti u smislu vremenske složenosti algoritama za rešavanje s-t problema najkraćeg puta, vremenska složenost Algoritma 2 može se, pored pomenutog konstantnog faktora, razlikovati za najviše faktor O(logn) u odnosu na optimalnu složenost po broju query-ja. Ovaj dodatni faktor proizlazi iz operacija nad redom sa prioritetom (heap-om) koji se koristi u implementaciji bidirekcionog Dijkstrinog algoritma.

Literatura:

Haeupler, B., Hladík, R., Rozhoň, V., Tarjan, R. E., & Tětek, J. (2025). Bidirectional Dijkstra's Algorithm is Instance-Optimal. Symposium on Simplicity in Algorithms (SOSA 2025).